

Day - 15: ADC & DAC

▼ Digital to Analog Converter | DAC Operation & Working

1. Introduction to DAC

- **Definition:** DAC stands for **Digital to Analog Converter**.
- **The Goal:** It takes digital data (binary bits) as an input and converts it into corresponding analog outputs, typically in the form of **voltage or current**.
- **Common Applications:** DACs are essential for producing sound and video outputs in embedded systems (like mobile phones, headphones, and speakers). They are also used to supply necessary voltage and current to control motors, or to actuate outputs for temperature and pressure controls.

2. The R-2R DAC Circuit Architecture

While DACs can be designed in various ways, most microcontrollers utilize an **R-2R Ladder Network**.

- **Hardware Components:** The circuit primarily consists of a resistor ladder (using resistors of value **R** and **2R**) and an Operational Amplifier (Op-Amp) used to provide necessary gain.
- **Core Requirements:** To function, the DAC block strictly requires a **Reference Voltage (V_{ref})** and a **Ground Voltage**.
- **Output Calculation:** The analog output voltage is calculated based on the input bits, the feedback resistance (**R_F**), the ladder resistance (**R**), and the reference voltage. The standard formula generally follows the structure:

$$V_{out} = -R_F * (V_{ref} / R) * (Binary Value / 2^N - 1).$$

3. Output Voltage Execution (4-Bit Example)

The video demonstrates calculating the output for a **4-bit DAC** (inputs **B0** to **B3**) with the following hardware parameters: Feedback Resistance (**R_F**) = 1 kΩ, Ladder Resistance (**R**) = 2 kΩ, and Reference Voltage (**V_{ref}**) = 5V.

- **Condition 1:** If all input bits are set to low (`0000`), the resulting analog output voltage is **0V**.
- **Condition 2:** If all input bits are set to high (`1111`), the resulting analog output voltage calculates to **4.68V**.
- *Note: In modern microcontrollers, developers do not calculate this manually; the program simply loads the input bits, and the hardware handles the continuous conversion.*

4. Understanding DAC Resolution

- **Definition:** Resolution is defined as the **smallest possible change in the analog output voltage or current** that the DAC hardware can physically produce.
- **The Formula:** $\text{Resolution} = V_{\text{ref}} / 2^N$ (where `N` equals the number of bits).

5. The Impact of Bit Size on Resolution

The overall quality of your analog output relies heavily on keeping the resolution value as small as possible.

- **4-Bit DAC Example:** Using the formula with a 5V reference, a 4-bit DAC ($2^4 = 16$) yields a resolution of **0.3125 Volts**. This means the output can only jump in steps of 0.3125V.
- **10-Bit DAC Example:** Using the same 5V reference, a 10-bit DAC yields a resolution of **4.8 millivolts**.
- **Conclusion:** Increasing the number of bits (`N`) actively reduces the resolution step value. A smaller resolution value provides a much smoother, clearer, and more accurate analog output. This is why modern 32-bit microcontrollers typically utilize **10-bit or 12-bit DACs/ADCs** to eliminate voltage fluctuations and produce high-quality audio and video.

▼ Square Wave Generation | DAC Programming

1. Objective & Hardware Overview

- **The Goal:** To write an Embedded C program that generates analog square and rectangular (pulse) waves using a Digital to Analog Converter (DAC).

- **Target Hardware:** The exercises are designed for the **LPC1768 microcontroller**, which features a built-in **10-bit DAC**.
- **Reference Voltage:** The DAC peripheral operates using a **3.3V** supply, meaning the maximum analog output voltage will be approximately 3.3V.

2. Waveform Definitions

By feeding specific binary inputs into the DAC, developers can construct various analog waveforms:

- **Square Wave:** A waveform where the high (ON) period and the low (OFF) period are exactly **equal** in duration.
- **Rectangular Wave (Pulse):** A waveform where the ON period and OFF period are **unequal** (e.g., a long ON period followed by a very short OFF period).

3. Essential Pins & Registers

To successfully program the DAC output, developers must manage specific pins and registers:

- **AOUT (Analog Output Pin):** The physical analog output of the DAC is mapped to pin **P0.26** on the microcontroller.
- **PINSEL1 Register:** To activate the **AOUT** capability, pin **P0.26** must be configured using the **PINSEL1** register. The code sets bits 20 and 21 to **Function 2**.
- **DACR (Digital to Analog Converter Register):** This is the exclusive register used to load data into the DAC. The digital input value must strictly be loaded into the specific window between **bit 6 and bit 15**.

4. Programming Implementation (Code Logic)

1. **Pin Configuration:** First, configure **P0.26** to operate as an analog output pin using the **PINSEL** macros.
2. **Define Values:** Define the high state as **0xFF** and the low state as **0x00**.
3. **10-Bit Conversion:** Because the defined high state (**0xFF**) is an 8-bit value and the LPC1768 has a 10-bit DAC, the value is **multiplied by 4** in the code to properly scale it into a 10-bit format.

4. **Load the Register:** The newly formatted 10-bit value is then pushed into the `DACR` register starting at bit 6 (using a left shift).

5. Generating the Specific Waves

- **For a Square Wave:** The code sets the DAC output to `0xFF` (high), calls a delay function (e.g., 100), sets the output to `0x00` (low), and calls the **exact same delay** (100). Because the delays are identical, it generates a perfectly symmetrical square wave.
- **For a Rectangular Wave:** The code sets the output high with a longer delay (e.g., 100), but sets the output low with a **shorter delay** (e.g., 20 or 30). This creates the unequal ON/OFF pulses characteristic of a rectangular wave.

6. Debugging & Verification

- **Logic Analyzer:** To verify the output without an external oscilloscope, developers use the **Logic Analyzer** tool built into the Keil software debug window.
- **Setup:** The signal is mapped by typing `AOUT` into the analyzer setup. Because the maximum reference voltage is 3.3V, the max display limit on the analyzer is set to 4 for better visibility.
- **Verification:** When the code runs, the logic analyzer actively visualizes the generated analog waveform, showing the voltage actively toggling between 0V and approximately 3.28V according to the programmed delay periods.

▼ Staircase Wave Generation | DAC Programming

1. Objective & Overview

- **The Goal:** To generate an analog staircase waveform using a DAC by providing it with specific binary inputs.
- **Core Logic:** Unlike a square wave that simply toggles between high and low, a staircase wave requires the program to send a series of distinctly

calculated digital values to the DAC over different time intervals to sequentially "step up" the voltage.

2. Calculating the Step Values

- **Resolution Limit:** The exercise assumes the use of an 8-bit DAC, meaning the digital input values can range from **0** to **255**.
- **The Math:** To determine the values for the steps, the programmer divides the maximum value (**255**) by the specific number of steps they want to generate.
- **Example:** To generate a waveform with 4 distinct steps, the calculation is 255 divided by 4, which yields approximately **64**. Therefore, each step will increase the digital input by 64.

3. The Output Sequence

Based on the calculations, the DAC is sequentially fed the following values to build the staircase shape:

- **Base:** **0**.
- **Step 1:** **64**.
- **Step 2:** **128**.
- **Step 3:** **192**.
- **Step 4 (Peak):** **255** (or **256**).
- **Cycle Reset:** After generating the highest step, the waveform can be programmed to either drop immediately back down to **0**, or decrease incrementally in reverse order to repeat the cycle.

4. Programming Implementation

- **Code Structure:** The foundational code structure is similar to generating a square wave, except the value assignment logic requires multiple blocks.
- **Execution Flow:** The code writes the first digital value (**0x00**) into the DAC control register and applies a standard delay. It then assigns the second value (**64**) to the register, applies a delay, and repeats this process sequentially up to **255**.

5. Debugging & Real-World Application

- **Verification:** Developers use a software debug window and a Logic Analyzer to run the compiled code. The analyzer will visually confirm the output, showing the analog voltage incrementally changing in steps (for example: 0V, 1.8V, 2.4V, 3.2V).
- **Why It Matters:** These generated staircase waveforms are not just experimental concepts; the varying analog output voltages serve as active control signals used to manage specific physical functionalities within real-world embedded system applications.

▼ Sine Wave Generation | DAC Programming

1. Objective & Overview

- **The Goal:** To write an Embedded C program that generates an analog sine wave using a Digital to Analog Converter (DAC).
- **Target Hardware:** The exercise uses the LPC1768 microcontroller, which features a **10-bit DAC**.
- **Core Concept:** Unlike basic square or rectangular waves, a sine wave is curved. To construct this curve, the programmer must calculate specific digital input values corresponding to different angles (e.g., 0°, 10°, 20°) and sequentially feed them to the DAC.

2. The Mathematical Formula

Because the DAC hardware cannot accept negative digital input values, the entire sine wave must be mathematically shifted above zero.

- **The Formula:** $\text{Digital Input} = 128 + (\sin(x) * 127)$.
- **x (Angle):** Represents the angle (theta). For this program, the angle is incremented in **10° intervals** from 0° up to 360°.
- **128 (Reference Midpoint):** Because negative values aren't possible, 128 acts as the new "zero" reference line or midpoint for the waveform.
- **127 (Amplitude):** This acts as the multiplier for the sine calculation.

3. Key Waveform Calculations

Using the formula above, the code calculates specific voltage points across a 360-degree cycle:

- **At 0°:** $\sin(0) = 0$. Calculation: $(0 * 127) + 128 = 128$ (Starting midpoint).
- **At 90°:** $\sin(90) = 1$. Calculation: $(1 * 127) + 128 = 255$ (The positive peak of the wave).
- **At 180°:** $\sin(180) = 0$. Calculation: $(0 * 127) + 128 = 128$ (Returns to midpoint).
- **At 270°:** $\sin(270) = -1$. Calculation: $(-1 * 127) + 128 = 1$ (Approximate to 0, acting as the negative peak).

4. Programming Implementation

- **Array Storage:** The math is not calculated live by the microcontroller. Instead, 36 specific data points (representing 0° to 360° in 10° steps) are pre-calculated and stored sequentially in an array.
- **Execution Loop:** Inside an infinite `while` loop, a `for` loop (`i = 0` to `<36`) iterates through the array.
- **10-Bit Conversion:** Because the calculated array values are based on an 8-bit scale (up to 255) and the microcontroller has a 10-bit DAC, each array element is **multiplied by 4** during the loop to scale it properly.
- **Data Loading:** Finally, this formatted value is loaded into the DAC control register, generating a single analog sample on the output.

5. Debugging & Precision Tuning

- **Verification:** The output is verified using the Logic Analyzer tool in the debug window by monitoring the **AOUT** (Analog Output) signal.
- **Improving the Waveform:** The demonstrated code generates a wave using a 10° step interval. To generate a much smoother and more precise sine wave, developers can decrease the angle step (for example, down to **5° intervals**) to provide more granular data samples to the DAC.

▼ Trapezoidal Signal Generation | DAC Programming

1. Objective & Overview

- **The Goal:** To write an Embedded C program that generates an analog trapezoidal waveform using a Digital to Analog Converter (DAC).
- **The Mechanism:** The program sequentially feeds specific digital input values to the DAC, which then produces the corresponding analog output voltages to physically construct the wave.

2. The Trapezoidal Waveform Structure

A trapezoidal wave is uniquely shaped by four distinct phases:

1. **Positive Ramp:** The voltage steadily increases.
2. **Constant High Voltage:** The voltage is maintained at its peak value (e.g., `255` or `0xFF`).
3. **Negative Ramp:** The voltage steadily decreases back down.
4. **Constant Zero Voltage:** The voltage is maintained at `0` before the cycle restarts.

3. Programming Implementation (Code Logic)

To generate these four specific phases, the programming logic relies on a combination of two `for` loops and specific time delays:

- **Phase 1 - Positive Ramp:** The program uses a first `for` loop to incrementally step up the values sent to the DAC, effectively building the upward slope (triangle wave shape).
- **Phase 2 - High Constant Voltage:** Once the first loop reaches the maximum value (`255`), the code executes a delay function (e.g., approximately 100 milliseconds). This forces the DAC to hold the analog output at that peak voltage, flattening the top of the wave.
- **Phase 3 - Negative Ramp:** A second `for` loop is then used to incrementally step the values back down, creating the downward slope.

- **Phase 4 - Low Constant Voltage:** When the downward loop hits 0, another delay (e.g., 100 milliseconds) is applied to hold the output steady at zero.

4. Continuous Execution & Verification

- **Continuous Generation:** By placing these two `for` loops and their respective delays inside an infinite loop, the microcontroller will continuously execute the sequence, generating an ongoing trapezoidal signal.
- **Verification:** After compiling the code, developers can use a **Logic Analyzer** tool to visually monitor the output. A successful program will clearly display the continuous cycle: positive ramp → constant high → negative ramp → constant low.

▼ Analog to Digital Converter | ADC Working

1. Introduction to ADC

- **Definition:** ADC stands for **Analog to Digital Converter**. It takes continuously varying analog inputs and converts them into digital or binary outputs.
- **The Problem:** In the physical world, sensors naturally capture analog data (such as temperature, pressure, humidity, and speed). However, microprocessors and memory units cannot read, process, or store analog information.
- **The Solution:** The ADC acts as an essential bridge, converting the analog sensor data into a digital format the processor can understand so it can execute appropriate embedded actions (like triggering an alarm or turning on a fan).

2. The Three Core Stages of ADC Conversion

Any ADC follows three sequential steps to convert a signal:

1. **Sampling:** The process of taking periodic "snapshots" of a continuous analog signal at a predefined time interval to create discrete signals.

- *Sampling Theorem*: The **Sampling Frequency (F_s)** must be greater than or equal to at least two times the frequency of the input analog message signal (F_m).
2. **Quantization**: The process of mapping and equating each of those sampled, discrete signals to its equivalent finite voltage level.
 3. **Encoding**: The final step where these mapped analog voltage levels are mathematically converted into their final digital/binary form.

3. Understanding ADC Resolution

- **Definition**: Resolution represents the **smallest possible change in the analog input voltage** that the ADC hardware is physically capable of detecting.
- **The Formula**: $\text{Resolution} = V_{\text{ref}} / 2^N$, where V_{ref} is the reference voltage and N is the number of bits the ADC uses.
- **The Impact of Bit Size**: A higher number of bits creates more discrete voltage levels, which actually results in a smaller resolution step. For example, using a 3.3V reference, an 8-bit ADC has a resolution of 12.8 mV, while a **10-bit ADC has a much more precise resolution of 3.22 mV**.

4. Types of ADCs & The SAR Architecture

- There are five main types of ADCs: **Successive Approximation Register (SAR)**, Sigma Delta, Dual Slope, and Pipeline.
- **SAR ADC**: This is the most popular type of ADC used in modern microcontrollers.

5. How the SAR ADC Works

The SAR ADC utilizes a few internal hardware blocks: a Sample and Hold circuit, a SAR unit, a DAC (Digital to Analog Converter), and a Comparator.

- **Step 1**: The continuous analog input is fed into the **Sample and Hold circuit** to capture a sampled analog voltage.
- **Step 2**: The SAR unit makes an initial digital guess by setting the **Most Significant Bit (MSB) to 1** and the rest of the bits to 0.

- **Step 3:** This digital guess is sent to the internal DAC, which immediately converts it into an analog voltage.
- **Step 4:** The **Comparator** checks the DAC's analog voltage against the actual sampled input voltage.
- **Step 5:** If the sampled input voltage is greater than the DAC voltage, the control logic **retains the MSB as 1**.
- **Step 6:** This trial-and-error process continues iteratively for every subsequent bit down the line until all combinations are tested.
- **Step 7:** Once complete, the SAR generates an **End of Conversion (EOC)** or "Done" bit and saves the final binary data into the ADC data register.

6. Real-World Applications

Because almost all real-world data is analog, ADCs are heavily utilized across industries:

- **Consumer Electronics:** Smartphones, digital cameras, and audio equipment.
- **Medical Instruments:** ECG machines, MRI scanners, and blood pressure/sugar monitors.
- **Industrial & Scientific:** Temperature, pressure, and vehicle speed monitoring systems, as well as precise laboratory measurements and voice communications.

▼ ADC Programming in Embedded C | LPC1768

1. Introduction & Hardware Setup

- **The Objective:** To develop Embedded C code that programs an Analog to Digital Converter (ADC) to read analog data (like from a temperature sensor) and convert it into a digital format.
- **Target Hardware:** The code is designed for the **LPC1768 microcontroller**, which is a 32-bit microcontroller equipped with a built-in **12-bit ADC**.
- **ADC Specifications:**

- The 12-bit resolution means the maximum digital output value it can produce is `0xFFF`.
- It operates at a frequency of 200 kHz.
- It is an **8-channel ADC**, meaning up to 8 different analog inputs (labeled AD0 to AD7) can be connected simultaneously.
- **Scenario Setup:** For this exercise, a temperature sensor is assumed to be connected to **Analog Input 2 (AIN2)**, which physically maps to microcontroller pin **P0.25**. The system uses a maximum reference voltage of **3.3V**.

2. Core ADC Registers

To configure and read from the ADC, the program must interact with specific registers:

- **ADCR (ADC Control Register):** This is the primary configuration register.
 - **SEL (Channel Selection):** An 8-bit field where each bit represents one channel. It tells the ADC which specific input channel to read from.
 - **CLKDIV (Clock Division):** Used to step down the microcontroller's peripheral clock to define the ADC's operating clock. In this code, it is divided by 23 to yield a 1 MHz ADC clock.
 - **Burst Mode (Bit 6):** Enables continuous analog-to-digital conversion without needing to constantly trigger it manually.
 - **PDN (Power Down Mode - Bit 21):** The ADC power is down by default; this bit must be explicitly enabled (turned ON) for the ADC to function.
- **ADDR (ADC Data Register):** The register where the final digital output is stored. Because there are 8 channels, there are dedicated data registers (like `ADDR0`, `ADDR2`) for each one.
 - **Result Field:** The actual 12-bit converted digital value is stored in bit positions **4 through 15**.
 - **DONE Bit:** A specific bit that gets set to indicate the "End of Conversion." It signals that the ADC has finished converting the data and it is ready to be read.

3. Programming Implementation

The coding architecture is modularized into two distinct functions:

A. ADC Configuration Function

1. **Power Enable:** First, enable power to the ADC peripheral.
2. **Pin Configuration:** Access the Pin Connect Block and modify the `PINSEL1` register. To activate the analog input function on pin `P0.25`, its function bits (positions 18 and 19) must be set to `01`.
3. **ADCR Setup:** Pass the configuration parameters to the `ADCR` register: select Channel 2, load the clock division value (23), turn Burst Mode ON, and turn the PDN bit ON.

B. ADC Conversion & Extraction Function

1. **Polling the DONE Bit:** The code uses a `while` loop to continuously check the Global Data Register (`ADGDR`) until the DONE bit is set, confirming the analog value has been fully converted.
2. **Data Masking:** Once ready, the program reads the `ADDR2` register. Since the 12-bit result is stored in bits 4 through 15, the code applies a bitwise AND mask to extract only those specific bits.
3. **Bit Shifting:** The extracted data is then physically right-shifted by 4 positions to align the bits cleanly as the Least Significant Bits (LSB), producing the final digital integer.

4. Debugging & Verification

- **Global Variables:** Since an LCD is not used in this demonstration, the final converted value is returned to the main program and stored in a global variable (e.g., `temp`).
- **Software Simulation:** The programmer can verify the operation using the Keil software's debug window and watch variables.
- **Testing Voltage Inputs:** By editing the simulated analog input voltage on Channel 2, the programmer can verify the output scales correctly.
 - At **1V, 1.5V, and 2.5V**, the digital variable dynamically updates to represent the varying values.

- At the absolute maximum input of **3.3V**, the ADC outputs the maximum 12-bit hexadecimal value of **FFF**.

▼ ADC Interrupt | ADC Interrupt Programming

1. Introduction & Objective

- **The Goal:** To generate a hardware interrupt from the Analog to Digital Converter (ADC) peripheral.
- **The Advantage:** Utilizing an interrupt completely avoids the inefficient **"polling" method**. Instead of forcing the processor to continuously wait and check if the data is ready, the processor can execute the main code and will only be interrupted exactly when the ADC data is fully converted and ready to be read.

2. The "Done" Bit (End of Conversion)

- When the ADC finishes converting analog data into digital data, it sets a specific bit known as the **End of Conversion (EOC)** or the **Done bit**.
- This specific Done bit acts as the trigger; whenever it is set, the ADC generates an interrupt signal, forcing the processor to jump to the Interrupt Service Routine (ISR) to handle the new data.

3. Interrupt Configuration Steps

To successfully generate an interrupt, the system must be configured in two main places:

1. **Standard Configuration:** The ADC must first be configured into its regular operating mode (like setting the clock division, enabling power, and selecting the channel).
2. **ADC Register Enable:** The interrupt must be explicitly enabled within the ADC's own control registers.
3. **NVIC Enable:** The interrupt must also be enabled inside the processor's **Nested Vectored Interrupt Controller (NVIC)**, which includes turning on the `ADC_IRQn` line and setting its priority.

4. Essential ADC Interrupt Registers

In this specific exercise, the code is written for **Analog Input Channel 2** (mapped to pin **P0.25**). To enable the interrupt for this channel, two specific bits must be set:

- **Channel Interrupt Enable (Bit 2):** Because the code uses Channel 2, bit number 2 in the ADC interrupt register must be enabled.
- **Global Interrupt Enable (Bit 8):** The ADC hardware requires the overarching global interrupt enable bit (bit number 8) to be set alongside the channel-specific bit.

5. The Interrupt Service Routine (ISR / Handler)

When the conversion is complete, the processor jumps to the ADC handler function, which executes the following steps:

- **Clear the Interrupt:** The very first action the ISR must take is to **read the ADC Status Register** to successfully clear the interrupt flag.
- **Read the Data:** The ISR reads the converted digital data from the ADC Data Register.
- **Format the Data:** The extracted data is logically right-shifted by 4 positions to format it cleanly as the Least Significant Bit (LSB).
- **Execute Action:** The formatted digital data is printed to the terminal, and the processor exits the ISR back to the main code.

6. Execution Flow & Debugging

- **Main Loop:** Inside the main code, a basic action is programmed—specifically, continuously toggling a GPIO pin (**P0.0**).
- **The Interruption:** Whenever an analog input is actively provided to Channel 2, the ADC converts it, triggers the interrupt, briefly pauses the main code (stopping the pin toggle), and executes the ISR to print the data to the terminal.
- **Verification:** By using a software debug window, a developer can toggle the global ADC interrupt ON and OFF to visually prove the execution flow. When the interrupt is disabled, the system continuously executes the main code and toggles the pin; when enabled, it successfully pauses to print the ADC readings.

